

SIH 2026 — Developer 3

Parallel Work + Post-Gateway Integration Plan

Application Backend / FastAPI • Phase 1

Purpose: give Developer 3 a clear execution plan for what can be completed immediately, what must wait for Developer 2's Blockchain Gateway/event bridge, and how the final three-microservice integration will be performed. This plan assumes the Phase 1 architecture already agreed for SIH 2026: Application Service (D3) orchestrates, Data Service (D1) provides the operational/read model, and Blockchain Service (D2) owns Fabric, chaincode, Gateway, and the Fabric event bridge.

1. Where We Are Now

Developer 2's Blockchain Phase 1 chaincode and Fabric transaction lifecycle has been implemented and verified. The verified work includes the Fabric network/channel, chaincode contracts, deterministic transaction timestamps, authorization/custody checks, transfer and receive flow, audit scan/verification through the AuditContract namespace, the RECEIVED → PROCESSED transition, and transformation testing. This blockchain chaincode work is now treated as the stable baseline. Developer 2's remaining Phase 1 integration responsibility is the Blockchain Gateway and the Gateway-to-Application Fabric event bridge.

Developer 1's Data Service has already been merged and includes the PostgreSQL operational/read model, Redis cache-aside layer, IPFS adapter, SQLAlchemy models/repositories, migrations, seed support, and integration tests. Developer 1's remaining Phase 1 integration work is to expose the missing real persistence contracts required by the Application Service, especially event retrieval and identity persistence.

Developer 3 therefore should not wait for Developer 2 to begin all work. The correct approach is to complete the Application Service structures and contracts that can be developed independently, while explicitly leaving the final Fabric-dependent integration for the point when the real Gateway and event bridge are available.

2. The Core Architecture Developer 3 Must Preserve

Layer	Owner	Primary responsibility	D3 boundary
Application Service	D3	FastAPI, validation, auth/RBAC, orchestration, API contract	D3 orchestrates; it does not become the database or blockchain
Blockchain Service	D2	Fabric network, chaincode, Gateway, Fabric event bridge	D3 calls the Gateway; D3 does not use Fabric SDK directly
Data Service	D1	PostgreSQL read model, Redis, IPFS, persistence APIs	D3 calls Data Service APIs; D3 does not directly access PostgreSQL
Client	External consumers	Uses public Application Service APIs	Client does not call Fabric/Data Service directly

3. What Developer 3 Can Start Immediately

The following work should start now. None of it requires a running Blockchain Gateway. Where an external service is not yet available, use mocks only inside tests and keep the real interface explicit. Do not create fake production behavior.

- Review the existing Application Service against the frozen Phase 1 API contract and identify the exact modules/routes that must be added or corrected.
- Implement the application-side Events API structure, including `getEvents` and `submitBusinessOperation`, using typed schemas and the Gateway/Data Service client boundaries.
- Implement the internal Fabric-event receiving endpoint according to the agreed event envelope. It must validate, authorize the internal source as appropriate, preserve Fabric transaction/event identifiers, and route recognized events to the Data Service.
- Implement the Blockchain Gateway client/adaptor in the Application Service. The adaptor must contain the Gateway URL, request/response schemas, timeout/error handling, and transaction correlation behavior rather than embedding raw Fabric logic in business services.
- Prepare the Application Service to call Developer 1's real Data Service contracts for event retrieval and organization/role persistence once those endpoints are available.
- Implement or complete `createOrganization` and `assignRole` at the application boundary with real authorization checks and real Data Service persistence; do not return fake 201 responses.
- Review product and batch write services and remove any architecture that treats an immediate synchronous PostgreSQL write after a Fabric call as the final source-of-truth flow.
- Create unit/contract tests for Gateway calls, webhook payload validation, event mapping, idempotency behavior, authorization, error mapping, and Data Service client calls.
- Document the exact interfaces D2 and D1 must satisfy so the final connection is mechanical rather than exploratory.

4. Work That Must NOT Be Pretended Complete Yet

Developer 3 must clearly distinguish local code/test readiness from real system integration. The following cannot be marked as fully complete until Developer 2 provides the actual Gateway/event bridge:

- A real Application → Blockchain Gateway → Fabric transaction path.
- A real committed Fabric event being emitted/forwarded by the Blockchain Service.
- A real Fabric event reaching POST /internal/fabric-events.
- A complete Fabric commit → Application event → Data Service → PostgreSQL synchronization test.
- A final end-to-end consistency proof comparing the trusted Fabric state with the PostgreSQL operational/read model.

5. Important: Do Not Use an Asyncio Queue as the Final Architecture

Do not solve the missing Gateway event bridge by inventing an in-process asyncio queue that makes the Application Service appear asynchronous. A non-durable in-memory queue is not the same thing as the committed Fabric event path defined by the architecture. The Application Service may use mocks or test doubles to verify its event-handling code, but production synchronization must ultimately be driven by the real Gateway/Fabric event.

6. Dependency With Developer 1

Developer 3 and Developer 1 can work in parallel. D3 should not wait for D1 to finish the entire Data Service. D3 should develop against the documented internal API contract and use mocked responses for isolated tests. Once D1 delivers the real GET /internal/events and identity persistence endpoints, D3 should replace the mock integration with the real Data Service client calls and run the contract/integration tests.

- D1 provides GET /internal/events and documents its query/response behavior.
- D1 provides real organization and role persistence endpoints and documents request/response/error behavior.
- D3 consumes those APIs through the Data Service client.
- Neither developer should directly access the other's database/code internals.

7. Dependency With Developer 2 — The Critical Wait Point

Developer 2 is building the missing Blockchain Gateway and event bridge. D3 can prepare everything around it, but the final integration must wait until D2 provides the actual interface. At that point D3 should connect the real Gateway URL and event source rather than redesigning the backend.

The intended final sequence is: Client → Application Service → Blockchain Gateway → Fabric transaction → Fabric commit → Gateway emits/forwards Fabric event → Application Service receives event → Application Service calls Data Service → PostgreSQL read model is updated → subsequent Application Service reads return the operational representation.

8. Exact Handoff D3 Should Request From D2

- Running Gateway base URL and health endpoint.
- Invoke endpoint: HTTP method, path, authentication, request body, response body, timeout and error mapping.
- Query endpoint(s): method, path, request and response formats.
- Fabric event delivery mechanism: webhook path, method, authentication, retry behavior and event envelope.
- Event types emitted by the Gateway and their entity identifiers.
- Fabric transaction ID field and event ID field.
- Expected behavior for duplicate/replayed events.
- A working sample of one committed product/batch event payload.
- A command or documented procedure for performing one real Gateway → Fabric → event delivery test.

9. Exact Handoff D3 Should Request From D1

- GET /internal/events contract, including filters/pagination if supported.
- Organization creation endpoint contract.
- Role assignment endpoint contract.
- Event persistence field mapping and Fabric transaction/event correlation fields.
- Authentication requirement for internal calls.
- Status codes and error response structure.
- Cache invalidation behavior for event/entity updates.

- A test command or integration proof showing the Data Service contract is operational.

10. Final Integration Test — When D2 Gateway Is Ready

After D2's Gateway and event bridge are ready, stop treating the services as isolated test targets and run one complete vertical business flow. The goal is to prove that the three microservices work as one system.

1. Call the Application Service to initiate a legitimate product/batch business operation.
2. Verify that D3 sends the operation to the Blockchain Gateway rather than directly to Fabric.
3. Verify the Gateway submits the transaction to Fabric and receives a successful commit/transaction result.
4. Verify the Gateway emits or forwards the committed Fabric event.
5. Verify D3 receives the event at its internal Fabric-event endpoint.
6. Verify D3 maps the event and calls D1's Data Service.
7. Verify D1 persists the resulting operational/read-model state in PostgreSQL.
8. Verify Redis invalidation/cache behavior remains correct.
9. Read the entity again through the Application Service and verify the returned operational state corresponds to the committed Fabric transaction.
10. Verify the Fabric transaction ID/event ID can be traced across Gateway → Application → Data Service.
11. Repeat an event delivery where appropriate and verify idempotency does not create duplicate records.

11. What D3 Should Not Do

- Do not modify Developer 2's chaincode to solve an Application Service integration problem.
- Do not add direct Fabric SDK access to the Application Service when the architecture requires the Gateway.
- Do not connect the Application Service directly to PostgreSQL.
- Do not make PostgreSQL the authoritative replacement for Fabric lifecycle state.
- Do not create fake production endpoints that return success without persistence or a real Gateway operation.
- Do not introduce Kafka, EPCIS, IoT ingestion, advanced streaming, or other Phase 2 infrastructure merely to close Phase 1.
- Do not perform unrelated refactoring while completing these integration contracts.

12. What 'Done' Means for Developer 3

- Application Service routes, schemas, authorization and orchestration required by Phase 1 are implemented.
- Gateway client/adaptor is implemented and tested against a defined contract.
- Fabric event receiver is implemented and tested with representative event payloads.
- `getEvents` and `submitBusinessOperation` are implemented.
- Organization and role operations use real Data Service persistence.
- Synchronous dual-write is not used as the final ledger/read-model consistency mechanism.
- Unit and contract tests pass without requiring the real Gateway.
- When D2 supplies the Gateway, the real end-to-end flow passes without architectural rewrites.
- No fake event source, fake persistence, direct DB access, or direct Fabric SDK bypass is introduced.

13. Message to Developer 3

You can start immediately. Developer 2 has completed and verified the Phase 1 Fabric/chaincode lifecycle, and that work will be committed as the stable blockchain baseline. Your immediate task is to prepare the Application Service integration layer around the frozen microservice contracts: Events API, Gateway adapter, Fabric-event receiver, organization/role API integration, authorization, schemas and tests. Developer 1 is working in parallel on the real Data Service contracts you will consume. Do not wait for either developer to finish their entire layer. However, do not mark the Fabric integration as complete until Developer 2 provides the real Gateway and event bridge. At that point, connect the real Gateway, perform the full vertical test, and verify Fabric commit → event → backend → Data Service → PostgreSQL read-model synchronization. This is the intended final Phase 1 integration.

14. Final Execution Model

Now	After D1/D2 interfaces are ready	Final proof
D3 builds API/Gateway adapter/webhook/tests with contracts and mocks.	D3 connects real D1 APIs and D2 Gateway.	Run complete API → Gateway → Fabric → event → D3 → D1 → PostgreSQL flow.
D1 builds missing persistence contracts and tests.	D1 runs real integration against D3 event delivery.	Verify persistent read model, cache behavior and event correlation.
D2 commits stable chaincode baseline and builds Gateway/event bridge.	D2 supplies Gateway URL + event contract + sample event.	Verify real Fabric commit and committed event delivery.

Important: This document is an execution/handoff plan for Phase 1 integration. It does not replace the PRD/TRD or change the agreed microservice ownership boundaries.